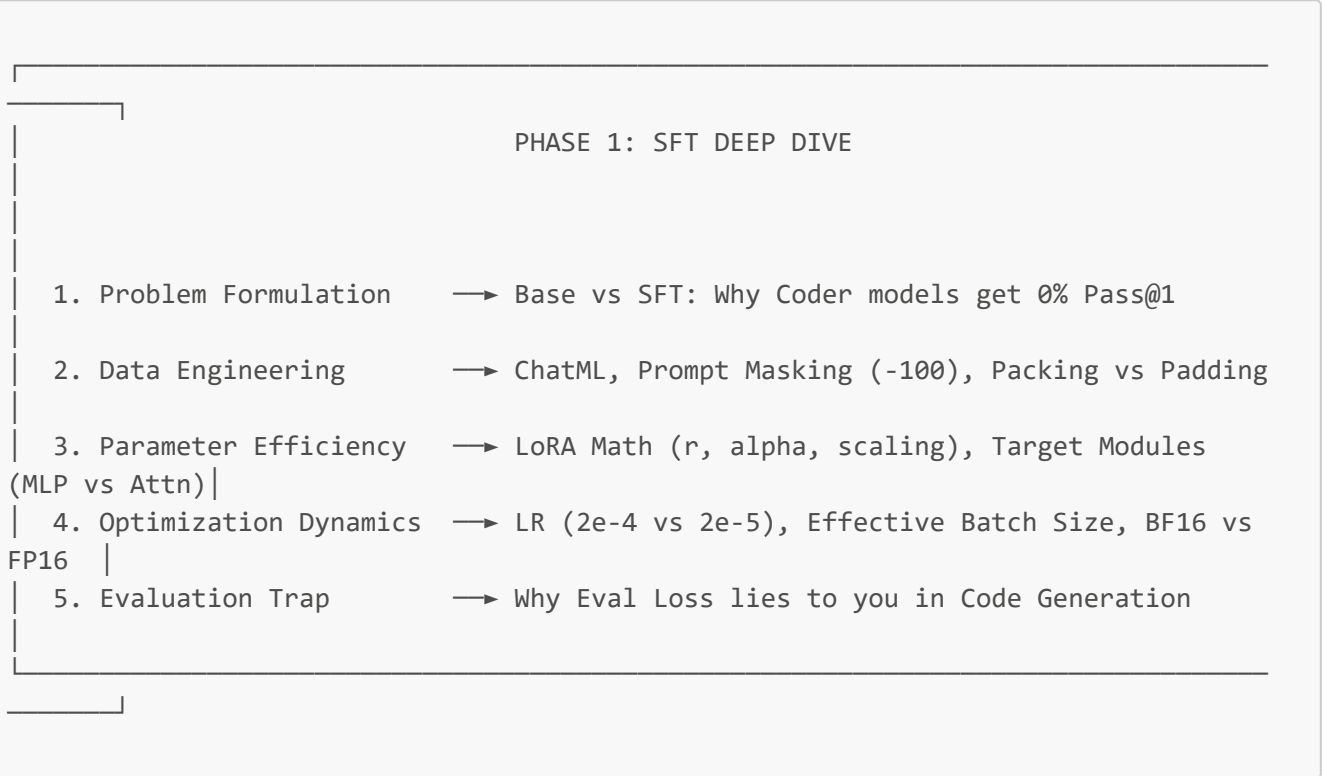
Phase 1 Deep Dive: SFT Mechanics, Recipe Space, & Engineering Decisions

Author: Monthir Ali
Companion File: `study-plan-post-training.md`
Target Topics: Data Templating, Prompt Loss Masking (`-100`), LoRA Mathematics (r, α, Target Modules), Optimization Dynamics, & The Evaluation Loss Trap in Code Generation.



---

1. Problem Formulation: Why Base Coder Models Get 0% Pass@1

`Qwen/Qwen2.5-Coder-1.5B-Instruct` is one of the strongest open code foundation models in the world. Yet, when evaluated on our held-out test suite of 150 experimental specifications, it scored **0.00% Pass@1 (0/150)** with **124** `ValueErrors`.

Why did this happen?

1. **Pre-training prior vs. Exact DSL constraints:** The base model understands generic Python and has seen general mentions of factorial experiment design and SAT solvers during pre-training. However, a Domain-Specific Language (DSL) like SweetPea has **strict syntactic and semantic invariants**:
   - `sp.CrossBlock(design, crossing, constraints)` requires `require_complete_crossing=False` if any level is excluded via `sp.Exclude`.
   - Between-trial transition predicates take relative dictionary offsets: `def is_repeat(t): return t[-1] == t[0]`.
   - Window derivations require explicit width and stride: `sp.Window(predicate, [factors], width, stride)`.

2. **Hallucinated Signatures:** The base model generated plausible-looking code (scoring ≈ 0.28 on stepped AST parsing) but invented method signatures like `sp.Factor.create()` or passed un-wrapped lambda functions to `CrossBlock`.

The Role of SFT: SFT is not teaching the model how to write Python from scratch; it is **steering the model's existing AST code-generation prior into the exact manifold of the SweetPea DSL compiler**.

---

2. Data Engineering & Formatting: The Silent Bugs of SFT

A. Chat Templating (ChatML)

Modern instruct models are trained with structured delimiters to distinguish system rules, user queries, and assistant outputs. For Qwen, the ChatML format is:

```
<|im_start|>system
You are an expert in experimental design using SweetPea...<|im_end|>
<|im_start|>user
Create a Stroop experiment crossing font color and text word...<|im_end|>
<|im_start|>assistant
```

B. Response-Only Loss Masking (The #1 SFT Mistake)

In standard autoregressive language modeling, cross-entropy loss is computed over every token $t_1, t_2, \ldots, t_N$:

$$\mathcal{L}_{\text{naive}} = -\frac{1}{N}\sum_{i=1}^{N} \log P(t_i \mid t_{<i})$$

Why naive training fails in SFT: If you pass the full conversation text to the loss function, the model calculates loss on the **system prompt and user prompt** as well as the assistant response.

- The model wastes gradient capacity learning to predict the user's phrasing.
- Training loss appears artificially low because predicting common natural language English prompts is easy.

The Fix (Implemented in `ResponseOnlyDataCollator`): We tokenize the full string, search for the assistant delimiter `<|im_start|>assistant\n`, and set the target labels for **all preceding tokens to `-100`**:

$$\text{labels}[i] = \begin{cases} -100 & \text{if } i < \text{assistant\_start\_idx or token is PAD} \\ \text{input\_ids}[i] & \text{if } i \geq \text{assistant\_start\_idx} \end{cases}$$

In PyTorch, `torch.nn.CrossEntropyLoss(ignore_index=-100)` automatically skips any index with value `-100` when computing both the loss and the backward gradients. The gradient update is computed **strictly on the generated Python tokens**.

C. Padding vs. Packing

- **Padding (`padding=True`):** Adds `<|pad|>` tokens so all sequences in a batch have equal length. Simple and clean, but wastes compute if sequence lengths vary wildly.
- **Packing (`packing=True`):** Concatenates multiple short examples into a single 1024 or 2048 token sequence separated by `<|im_end|>`.
  - *Advantage:* Up to $2 \times -3\times$ training throughput.
  - *Risk (Attention Leakage):* Unless you use FlashAttention with custom block-diagonal attention masks, tokens from Example B can attend to tokens from Example A in the same window.
  - *Our Decision:* Because our SweetPea dataset has consistent sequence lengths (~250–400 tokens), batched padding with response masking provided 100% clean separation without attention leakage.

---

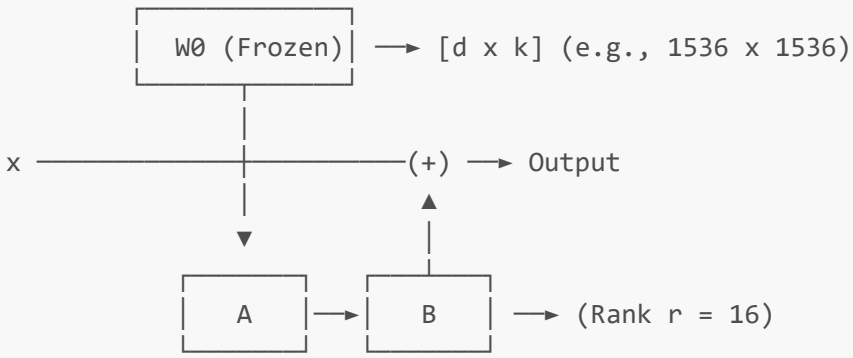
3. LoRA Mechanics: Rank (r), Alpha (α), and Target Modules

Full fine-tuning updates all 1.56B parameters ($W \leftarrow W + \Delta W$), requiring optimizer states (AdamW stores 2 states per parameter: momentum and variance in 32-bit float $\rightarrow 12\ \text{GB}$ of optimizer memory alone).

LoRA freezes $W_0 \in \mathbb{R}^{d \times k}$ and decomposes the update into two low-rank matrices:

$$\Delta W = \frac{\alpha}{r}(B \cdot A)$$

where $A \in \mathbb{R}^{r \times k}$ is initialized with Gaussian noise $\mathcal{N}(0, \sigma^2)$ and $B \in \mathbb{R}^{d \times r}$ is initialized to 0 (ensuring $\Delta W = 0$ at step 0).



Hyperparameter Choices & Rationale

Parameter	Value	Why this value?
Rank (r)	`16`	For simple style transfer, $r = 4$ or $r = 8$ suffices. For code synthesis and AST translation, $r = 16$ provides enough rank capacity to learn multiple constraint paradigms without risk of overfitting.
Alpha (α)	`32`	α is a constant scaling factor. The effective update is scaled by $\frac{\alpha}{r} = \frac{32}{16} = 2.0$. **Rule of thumb:** Always set $\alpha = 2 \times r$ so that when you change r in experiments, your effective learning rate does not unintentionally drift.

Parameter	Value	Why this value?
Dropout	`0.05`	Small regularization on the low-rank activations to prevent adapter co-adaptation on synthetic templates.
Target Modules	**All Linear**	Targeted `["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`.

The "All Linear" vs "Attention Only" Decision

The original 2021 LoRA paper only applied adapters to W_q and W_v. Modern post-training literature (including Dettmers et al., *QLoRA*) showed that:

1. **Attention layers (`q, k, v, o`)** control *where to look* (retrieving information across the prompt).
2. **MLP layers (`gate, up, down`)** store *factual associations and syntactic knowledge*.
3. Targeting all linear layers requires only marginally more parameters (18.4M params, 1.18% of the model), but is vastly superior for learning domain DSL syntax.

---

4. Optimization Dynamics: Learning Rates, Batches, and Schedules

A. Learning Rate: Why 2×10^{-4}?

- For **Full Fine-Tuning**, the standard learning rate is small: 1×10^{-5} to 3×10^{-5}. Higher rates destroy the pre-trained weights.
- For **LoRA**, the learning rate is $10\times$ **higher** (1×10^{-4} to 3×10^{-4}).
- *Reason:* W_0 is frozen. Only the adapter matrices A and B receive gradients. Because B starts at zero, the adapter needs a larger step size to establish strong features in the low-rank subspace.

B. Effective Batch Size & Gradient Accumulation

We configured:

- `per_device_train_batch_size = 4`
- `gradient_accumulation_steps = 4`
- **Effective Batch Size =** $4 \times 4 = 16$

The trade-off:

- A batch size of 1 is too noisy: individual examples cause large gradient spikes that degrade generalization.
- A batch size of 64 or 128 requires massive VRAM and takes fewer steps per epoch, which can under-fit on small datasets.
- An effective batch size of 16 is the standard sweet spot for code SFT.

C. Warmup & Cosine Decay

- **Warmup (10 steps):** Gradually ramps the learning rate from $0 \to 2 \times 10^{-4}$. This prevents massive gradient norms in the very first few steps from destabilizing the randomly initialized matrix A.
- **Cosine Annealing:** Smoothly decays the learning rate to near-zero (1.13×10^{-6} at step 114). This allows the optimizer to settle into a sharp local minimum at the end of training.

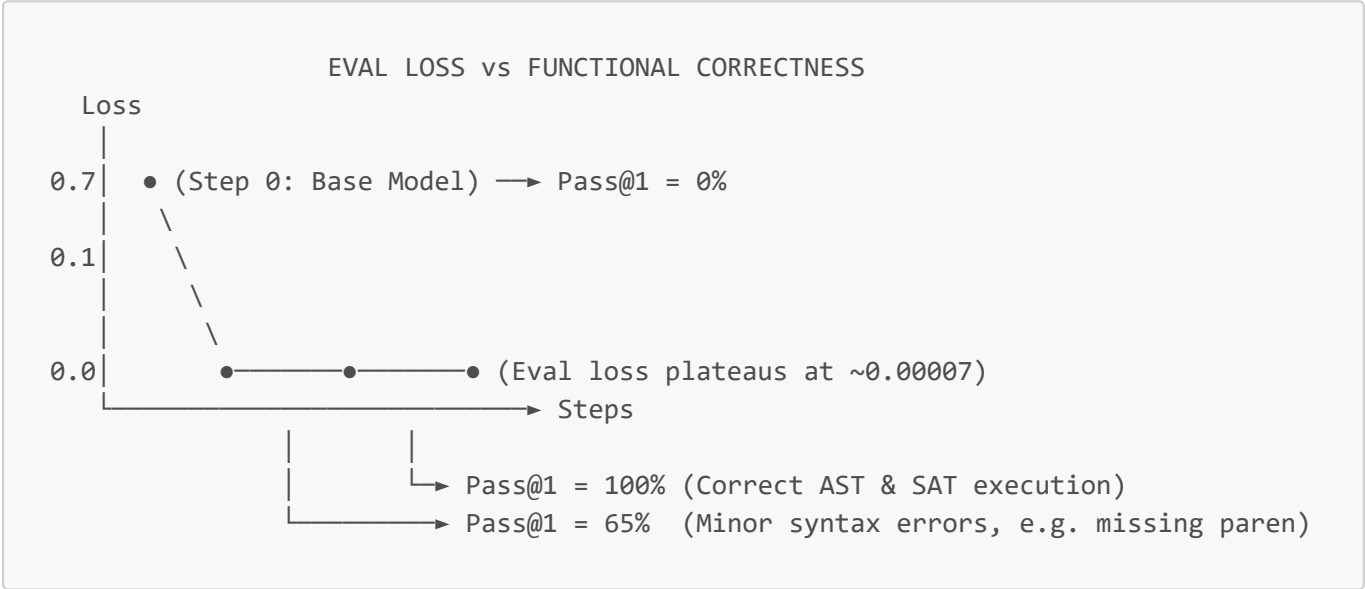
D. Precision: BF16 vs FP16 on Ada Lovelace (RTX 4080)

- **FP16 (IEEE Half-Precision):** 5 exponent bits, 10 mantissa bits. Small dynamic range ($\approx 6.5 \times 10^4$). Frequently suffers from gradient underflow/overflow, requiring complex dynamic loss scaling (`GradScaler`).
- **BF16 (Bfloat16):** 8 exponent bits (same dynamic range as FP32), 7 mantissa bits. Eliminates gradient underflow completely. Because the RTX 4080 (Ada architecture) has native BF16 Tensor Cores, BF16 is faster, more stable, and requires no loss scaler.

---

5. The Critical Post-Training Trap: When Eval Loss is Lying to You

In standard NLP benchmarks, practitioners monitor Cross-Entropy Evaluation Loss. **In code generation, eval loss can be actively deceptive:**



Why Eval Loss Lies:

1. **The "Single Character Syntax Failure" Problem:** If a model outputs 300 tokens of Python code and makes **one typo** in a keyword (`sp.CrossBlok` instead of `sp.CrossBlock`), 299 tokens are correct. Cross-entropy loss will be extremely low (≈ 0.005), yet compiler **Pass@1 is 0.0%**.
2. **The "Valid Paraphrasing" Problem:** The validation set has one canonical target solution. If the trained model writes equivalent, perfectly compiling code with different variable names or alternative factor ordering, its cross-entropy loss will *increase* (penalized for not matching the exact string), while functional compiler correctness is 100%.

Our Architecture Solution: We built `src/compiler_grader.py` and `src/evaluate.py` to evaluate **compiler execution Pass@1** directly with CryptoMiniSat alongside token loss.

---

6. Summary of Phase 1 Codebase Components

Component	Path	Functionality
Templates	`data/sweetpea_templates.py`	Generates 7 cognitive science experiment paradigms (Stroop, Flanker, Task-switching, N-back, Constrained

Component	Path	Functionality
		crossings).
Dataset Engine	`data/generate_dataset.py`	Synthesizes verified datasets (`train.jsonl`, `val.jsonl`, `test_heldout.jsonl`) with CryptoMiniSat checks.
Compiler Sandbox	`src/compiler_grader.py`	Isolated subprocess grader with AST validation, timeout guards, and stepped scoring.
SFT Training Loop	`src/train_sft.py`	LoRA SFT with custom `ResponseOnlyDataCollator` (masking prompt tokens to `-100`).
Evaluation Engine	`src/evaluate.py`	Batched GPU generation + parallel multiprocessing compiler grader.
SFT Config	`configs/sft_qwen_1.5b.yaml`	Reproducible configuration specifying LoRA rank, alpha, batch size, learning rate schedule.